
CHAPITRE 3

Algorithmes élémentaires et complexité

Écrire, tester et comparer plusieurs façons de résoudre un problème

Le travail du chapitre

Deux programmes peuvent donner le même résultat en faisant des quantités de travail très différentes. Nous allons écrire des versions simples, les tester, puis chercher ce que l'on peut éviter de refaire.

- Recherche d'une valeur, détection d'un doublon, deux prix pour un budget.
- Moyennes sur des fenêtres voisines et résumé d'un journal d'événements.

Pour les activités de code : quelques lignes de pseudocode, une trace sur les données proposées, puis le programme et ses tests. Les corrections suivent les essais.

Gardez vos fonctions dans le même notebook ou fichier Python : nous les réutiliserons. À chaque comparaison, précisez ce qui grandit, le travail répété et la mémoire ajoutée.

Un premier compteur

À vous. Prévoyez les deux valeurs renvoyées par cette fonction. Tracez `x`, `positifs` et `tests` à la fin de chaque tour, puis lancez le code.

```
def compter_positifs(valeurs):  
    positifs = 0  
    tests = 0  
    for x in valeurs:  
        tests = tests + 1  
        if x > 0:  
            positifs = positifs + 1  
    return positifs, tests  
  
print(compter_positifs([3, -1, 0, 4, 2]))
```

Que devient le nombre de tests si la liste contient 100 valeurs ? Et si toutes sont négatives ?

Le nombre de résultats et le travail effectué

Au départ, les deux compteurs valent 0.

Tour	x	positifs après	tests après
1	3	1	1
2	-1	1	2
3	0	1	3
4	4	2	4
5	2	3	5

L'affichage est (3, 5) : trois valeurs positives, mais cinq valeurs examinées.

Pour n éléments, on effectue exactement n tests $x > 0$, même si aucun élément n'est positif. La boucle s'arrête quand toute la liste a été parcourue. La liste vide donne (0, 0).

Ce que mesure la complexité

La taille de l'entrée décrit ce qui grandit : $n = \text{len}(\text{valeurs})$ pour une liste ; n et m pour deux listes indépendantes ; n et q pour un catalogue et ses requêtes.

Le temps décrit le nombre d'opérations lorsque ces tailles augmentent. Un parcours qui fait un travail borné par élément est en $O(n)$: doubler n double approximativement le travail. L'écriture $O(\cdot)$ donne une borne de croissance, pas un nombre de secondes.

Notre modèle : les nombres et identifiants sont de taille bornée ; comparer deux valeurs coûte $O(1)$. Les très grands entiers et les longues chaînes demanderaient de compter aussi leur taille.

Nous préciserons le **pire cas**, ou le **coût moyen** lorsqu'il s'agit du hachage. Une mesure sur une machine complète cette analyse ; elle ne la remplace pas.

Deux boucles : combien de tours ?

À vous. Avec $n = 3$ et $m = 2$, comptez les incrémentations de `total`. Écrivez les couples (i, j) parcourus par B.

A : l'une après l'autre

```
n, m = 3, 2
total = 0
for i in range(n):
    total = total + 1
for j in range(m):
    total = total + 1
print(total)
```

B : l'une dans l'autre

```
n, m = 3, 2
total = 0
for i in range(n):
    for j in range(m):
        total = total + 1
print(total)
```

Donnez ensuite une expression avec n et m . Que se passe-t-il si $m = n$?

Additionner ou multiplier les répétitions

	A : successives	B : imbriquées
Pour $n = 3, m = 2$	5 incrémentations	6 incrémentations
Pour n et m	$n + m$	$n \times m$
Si $m = n$	$2n$, donc $O(n)$	n^2 , donc $O(n^2)$

Dans B, les couples parcourus sont :

$(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1)$.

La boucle interne recommence pour chaque i . Dans A, le second parcours ne recommence pas.

Un nombre fixe de parcours, comme $3n + 20$ opérations, reste en $O(n)$. Si n et m sont indépendants, on conserve les deux tailles dans l'analyse.

Une recherche cachée dans une ligne

À vous. Ce programme ne montre qu'une boucle. Est-ce suffisant pour conclure qu'il est en $O(n)$?

```
def compter_presents(demandes, catalogue):  
    total = 0  
    for valeur in demandes:  
        if valeur in catalogue:  
            total = total + 1  
    return total  
  
print(compter_presents([9, 8, 7], [1, 2]))
```

Les deux arguments sont des **listes**. Sur cet exemple, comptez les comparaisons de valeurs nécessaires aux trois tests d'appartenance.

Puis posez $n = \text{len}(\text{demandes})$ et $m = \text{len}(\text{catalogue})$. Quel est le pire cas ?

Le coût du test d'appartenance

L'affichage est 0. Chaque valeur demandée est absente du catalogue.

Valeur cherchée	Valeurs comparées	Comparaisons
9	1, puis 2	2
8	1, puis 2	2
7	1, puis 2	2

On effectue 6 comparaisons. Dans le pire cas, chaque test valeur in catalogue parcourt les m valeurs : le travail peut atteindre $n \times m$ comparaisons, donc $O(nm)$.

Le coût d'une instruction dépend de l'objet utilisé. Avec un ensemble, l'appartenance a un coût moyen $O(1)$; il faudra aussi compter la construction et la mémoire de cet ensemble.

Quelques ordres de croissance

Classe	Exemple	Quand n double
$O(1)$	Lire une position dans une liste	Coût borné, indépendant de n
$O(\log n)$	Diviser la zone de recherche par deux	Environ une étape de plus
$O(n)$	Parcourir chaque élément	Environ deux fois le travail
$O(n \log n)$	Trier par comparaison, borne usuelle	Un peu plus de deux fois
$O(n^2)$	Comparer toutes les paires	Environ quatre fois

Pour $n = 100\,000$: $n = 10^5$, $n \log_2 n \approx 1,66 \times 10^6$, $n^2 = 10^{10}$. Ce sont des **ordres de grandeur de travail**, sans unité de temps.

La classe de croissance décrit l'évolution. Les constantes, les données et la machine expliquent aussi les durées réelles.

La mémoire créée pendant le calcul

À vous. On reçoit une liste de nombres et on renvoie ses valeurs strictement positives.

```
def garder_positifs(valeurs):  
    resultat = []  
    for x in valeurs:  
        if x > 0:  
            resultat.append(x)  
    return resultat
```

Pour $[3, -1, 2]$, donnez la sortie et son contenu après chaque tour. Puis distinguez ce qui existe déjà à l'entrée, ce que la fonction doit renvoyer et ce qu'elle utilise en plus.

Si les n valeurs sont positives, quelles quantités de mémoire augmentent avec n ?

Entrée, sortie et espace auxiliaire

La liste `resultat` vaut successivement `[3]`, `[3]`, puis `[3, 2]`.

Catégorie	Dans cette fonction	Au maximum
Entrée reçue	valeurs	n éléments déjà présents
Sortie construite	<code>resultat</code>	n références : $O(n)$
Auxiliaire hors sortie	Variable <code>x</code> , état du parcours	$O(1)$

Convention du chapitre : l'espace auxiliaire exclut l'entrée et la sortie obligatoire. La mémoire supplémentaire *totale*, sortie comprise, est ici $O(n)$.

On compte ce qui coexiste au pic d'exécution. Créer puis abandonner une tranche à chaque tour ne signifie pas que toutes ces tranches restent en mémoire simultanément.

Recherche linéaire : le premier indice

À vous. Écrivez `chercher_lineaire(valeurs, cible)`.

- La liste contient des entiers, dans un ordre quelconque.
- La fonction renvoie l'indice de la première apparition de la cible, ou `None` si elle est absente.
- Elle ne modifie pas la liste. N'utilisez ni `index()` ni recherche avec `in`.

```
valeurs = [2, 5, 8, 12, 16, 23, 38]
# Prévoir les résultats, puis écrire la fonction.
# chercher_lineaire(valeurs, 12)
# chercher_lineaire(valeurs, 6)
# chercher_lineaire([], 6)
```

Dans le pseudocode, indiquez quand on peut s'arrêter. Tracez les indices visités pour la cible 12.

Recherche linéaire : correction

```
def chercher_lineaire(valeurs, cible):  
    for i in range(len(valeurs)):  
        if valeurs[i] == cible:  
            return i  
    return None
```

```
valeurs = [2, 5, 8, 12, 16, 23, 38]  
print(chercher_lineaire(valeurs, 12))  
print(chercher_lineaire(valeurs, 6))  
print(chercher_lineaire([], 6))
```

Affichages : 3, puis None, puis None.

Pour 12, les indices visités sont 0, 1, 2, 3 ; le return arrête l'appel. Meilleur cas : une valeur examinée. Pire cas : n valeurs, donc $O(n)$ en temps et $O(1)$ auxiliaire.

Recherche dichotomique : un essai à la main

La liste est maintenant garantie **triée dans l'ordre croissant**. On regarde la valeur du milieu de l'intervalle encore possible.

Indice	0	1	2	3	4	5	6
Valeur	2	5	8	12	16	23	38

À vous. Cherchez 23, puis 6. À chaque étape, notez les bornes incluses gauche, droite, puis milieu = (gauche + droite) // 2 et la valeur consultée.

- Si la cible est plus grande, quelle moitié peut-on écarter ?
- Si elle est plus petite, quelle moitié reste possible ?
- Quel état des bornes signifie qu'il ne reste plus de position ?

Ne codez pas encore : justifiez les positions éliminées grâce au tri.

La réduction de l'intervalle : correction

Cible	Gauche	Droite	Milieu	Valeur	Décision
23	0	6	3	12	Chercher entre 4 et 6.
23	4	6	5	23	Renvoyer 5.
6	0	6	3	12	Chercher entre 0 et 2.
6	0	2	1	5	Chercher entre 2 et 2.
6	2	2	2	8	La borne droite devient 1.

Pour 6, gauche = 2 et droite = 1 : l'intervalle est vide, la cible est absente.

Ce que l'on conserve à chaque tour : si la cible existe, au moins une de ses occurrences reste dans l'intervalle courant. C'est l'**invariant** de recherche. Sans le tri, une valeur écartée pourrait encore être la cible.

La dichotomie en Python

À vous. Écrivez `chercher_dicho(valeurs, cible)` à partir de la trace précédente. La fonction renvoie un indice trouvé, ou `None`. S'il existe plusieurs occurrences, n'importe laquelle convient.

- Au départ, `gauche = 0` et `droite = len(valeurs) - 1`.
- Les deux bornes sont incluses : une seule position restante doit encore être examinée.
- Une comparaison qui échoue doit exclure le milieu déjà testé.

```
# Vérifier la liste vide, une seule valeur,  
# la première valeur, la dernière et une cible absente.  
# chercher_dicho([], 5)  
# chercher_dicho([5], 5)  
# chercher_dicho([2, 5], 5)  
# chercher_dicho([2, 5], 4)
```

Le tri est une précondition. La fonction n'a pas à trier ni à vérifier tout le tableau à chaque appel.

La dichotomie : correction

```
def chercher_dicho(valeurs, cible):  
    gauche = 0  
    droite = len(valeurs) - 1  
    while gauche <= droite:  
        milieu = (gauche + droite) // 2  
        if valeurs[milieu] == cible:  
            return milieu  
        if valeurs[milieu] < cible:  
            gauche = milieu + 1  
        else:  
            droite = milieu - 1  
    return None
```

Chaque tour réduit l'intervalle, même quand il ne reste qu'une position. Temps $O(\log n)$ pour $n \geq 2$; espace auxiliaire $O(1)$.

Les tests de la dichotomie

```
assert chercher_dicho([], 5) is None
assert chercher_dicho([5], 5) == 0
assert chercher_dicho([5], 4) is None
assert chercher_dicho([2, 5], 2) == 0
assert chercher_dicho([2, 5], 5) == 1
assert chercher_dicho([2, 5], 4) is None
```

Les six assertions passent sans affichage.

- La liste vide donne immédiatement un intervalle vide : `gauche = 0`, `droite = -1`.
- Une liste d'un élément exige de tester le cas `gauche == droite`.
- Les deux extrémités doivent rester accessibles. Une cible absente termine avec `gauche > droite`.

Ces petits cas font souvent apparaître une erreur de borne avant les grandes listes.

Une borne qui ne bouge plus

À vous. Dans la fonction précédente, on remplace seulement cette instruction :

```
gauche = milieu + 1
```

par :

```
gauche = milieu
```

Sur $[2, 5]$, avec la cible 5, faites deux tours à la main. Notez gauche, droite, milieu et valeurs[milieu].

Pourquoi le programme ne termine-t-il plus ? Quelle correction restaure la progression ? Gardez cette version sur papier pour repérer le blocage avant de l'exécuter.

La terminaison de la recherche

Tour	Gauche	Droite	Milieu	Conséquence
1	0	1	0	$2 < 5$, donc gauche = 0.
2	0	1	0	Même état, même décision.

Le milieu a déjà été testé et ne contient pas la cible. La bonne mise à jour est gauche = milieu + 1. La recherche passe alors à l'indice 1 et renvoie 1.

De l'autre côté, on utilise droite = milieu - 1. La quantité droite - gauche + 1 diminue tant que l'intervalle est non vide.

Sur $n \geq 1$ positions, on consulte au plus $\lfloor \log_2 n \rfloor + 1$ valeurs. Pour un million de positions, cela fait au plus 20 valeurs consultées, une fois la liste triée.

Un doublon parmi toutes les paires

À vous. Écrivez `doublon_paires(valeurs)` pour une liste d'entiers. La fonction renvoie `True` si deux positions distinctes contiennent la même valeur, sinon `False`. Elle ne modifie pas la liste.

Comparez les paires d'indices avec $i < j$: on évite de comparer une position avec elle-même et de vérifier deux fois la même paire.

```
# Prévoir le résultat, puis écrire deux boucles.  
# doublon_paires([4, 1, 7, 1])  
# doublon_paires([4, 1, 7, 9])  
# doublon_paires([])
```

Listez les paires testées jusqu'au premier doublon sur `[4, 1, 7, 1]`. Combien de paires faut-il examiner si les quatre valeurs sont distinctes ?

Les paires : correction

```
def doublon_paires(valeurs):  
    for i in range(len(valeurs)):  
        for j in range(i + 1, len(valeurs)):  
            if valeurs[i] == valeurs[j]:  
                return True  
    return False  
  
print(doublon_paires([4, 1, 7, 1]))  
print(doublon_paires([4, 1, 7, 9]))  
print(doublon_paires([]))
```

Affichages : True, puis False, puis False.

Sur le premier exemple : (0,1), (0,2), (0,3), (1,2), puis (1,3). La cinquième comparaison trouve le doublon. Sans doublon, on compare aussi (2,3).

Le nombre de paires à comparer

Dans le pire cas, toutes les valeurs sont distinctes et aucun retour anticipé n'est possible.

Taille n	Paires distinctes $n(n-1)/2$
4	6
10	45
100	4 950
1 000	499 500

Pour $i = 0$, on teste $n - 1$ partenaires ; pour $i = 1$, $n - 2$; puis jusqu'à 1. Le total est $(n - 1) + (n - 2) + \dots + 1 = n(n - 1)/2$, donc $O(n^2)$.

Les deux indices occupent un espace auxiliaire $O(1)$. On va garder le même résultat attendu et remplacer les comparaisons répétées.

Des doublons côte à côte après un tri

À vous. Écrivez une deuxième version, `doublon_tri(valeurs)`. Utilisez `sorted(valeurs)`, puis comparez seulement les voisins de la copie triée.

- Prévoyez la copie obtenue à partir de `[4, 1, 7, 1]`.
- Quel est le premier indice où l'on peut comparer un élément à son précédent ?
- Pourquoi la liste d'origine reste-t-elle intacte ?

```
valeurs = [4, 1, 7, 1]
# Écrire doublon_tri et vérifier le booléen renvoyé.
# Vérifier aussi que valeurs est toujours [4, 1, 7, 1].
```

Dans votre analyse, comptez le tri avant le parcours et la mémoire de la copie.

Le tri suivi d'un parcours : correction

```
def doublon_tri(valeurs):  
    copie = sorted(valeurs)  
    for i in range(1, len(copie)):  
        if copie[i] == copie[i - 1]:  
            return True  
    return False  
  
valeurs = [4, 1, 7, 1]  
print(doublon_tri(valeurs))  
print(valeurs)
```

Affichages : True, puis [4, 1, 7, 1].

La copie est [1, 1, 4, 7]. La première comparaison entre voisins suffit ici. Pire cas : tri en $O(n \log n)$, puis parcours en $O(n)$; espace auxiliaire $O(n)$.

Source : Documentation Python : Sorting Techniques

Se souvenir des valeurs déjà rencontrées

À vous. Écrivez `doublon_ensemble(valeurs)` en un seul parcours. Un ensemble `vus` doit contenir uniquement les valeurs rencontrées *avant* la valeur courante.

Commencez par le pseudocode. Pour chaque valeur, que faut-il faire d'abord : tester sa présence ou l'ajouter à `vus` ? Justifiez avec une liste d'un seul élément.

Tracez la fonction pour `[4, 1, 7, 1]` avec les colonnes :

Valeur courante	Ensemble avant	Déjà présente ?	Suite
4	<code>set()</code>	...	...
...	...	...	...

Les éléments sont des entiers : ils peuvent servir d'éléments d'un ensemble.

Le test avant l'ajout : correction

```
def doublon_ensemble(valeurs):  
    vus = set()  
    for valeur in valeurs:  
        if valeur in vus:  
            return True  
        vus.add(valeur)  
    return False
```

Valeur	Ensemble avant	Présente ?	Suite
4	set()	Non	Ajouter 4.
1	{4}	Non	Ajouter 1.
7	{4, 1}	Non	Ajouter 7.
1	{4, 1, 7}	Oui	Renvoyer True.

L'appel renvoie True. Ajouter avant de tester ferait prendre la première valeur pour un doublon.

Trois solutions au même problème

Méthode	Temps	Auxiliaire	Pourquoi ?
Toutes les paires	$O(n^2)$ au pire	$O(1)$	Comparaisons répétées
Copie triée, puis voisins	$O(n \log n)$ au pire	$O(n)$	Coût du tri et de la copie
Ensemble des valeurs vues	$O(n)$ en moyenne	$O(n)$ au pire	Appartenance moyenne $O(1)$

Les trois versions renvoient un booléen et conservent l'entrée. L'ensemble supprime la recherche parmi toutes les valeurs précédentes, en échange de mémoire.

`list.sort()` modifierait l'entrée. De plus, le tri de Python peut utiliser une mémoire de travail en $O(n)$: « sur place » ne garantit pas un espace auxiliaire $O(1)$.

Source : CPython : notes sur l'algorithme de tri

Les coûts des opérations utilisées

Opération	Coût	Travail à compter
<code>L[i], len(L)</code>	$O(1)$	Accès direct
<code>x in L</code>	$O(n)$ au pire	Parcours jusqu'à la valeur ou la fin
<code>L.append(x)</code>	$O(1)$ amorti	Agrandissements occasionnels
<code>L.insert(0, x)</code>	$O(n)$	Décalage des éléments
<code>L[a:b]</code>	$O(k)$	Copie de k références
<code>x in ensemble, cle in d</code>	$O(1)$ en moyenne	Hachage

Amorti : le coût de rares agrandissements se répartit sur une suite d'ajouts. **Moyen** : le comportement usuel du hachage ; certaines collisions peuvent coûter davantage. Les éléments et clés doivent être hachables. Ces coûts supposent des valeurs de taille bornée.

Plusieurs questions sur le même catalogue

À vous. Cette fonction répond à une liste de questions d'appartenance.

```
def repondre_scans(identifiants, requetes):  
    reponses = []  
    for cible in requetes:  
        reponses.append(cible in identifiants)  
    return reponses  
  
print(repondre_scans([10, 20, 30], [30, 99, 10]))
```

Prévoyez la sortie, puis écrivez `repondre_index` en construisant un ensemble **une seule fois**. Conservez l'ordre des réponses, même si le catalogue devient un ensemble.

Posez $n = \text{len}(\text{identifiants})$, $q = \text{len}(\text{requetes})$. Quel travail recommencerait-on en plaçant `set(identifiants)` dans la boucle ?

Un index construit une seule fois

```
def repondre_index(identifiants, requetes):  
    index = set(identifiants)  
    reponses = []  
    for cible in requetes:  
        reponses.append(cible in index)  
    return reponses  
  
print(repondre_index([10, 20, 30], [30, 99, 10]))
```

Résultat

```
[True, False, True]
```

La construction coûte $O(n)$ en moyenne, puis les q questions coûtent $O(q)$ en moyenne : total $O(n + q)$. L'index utilise $O(n)$ auxiliaire ; la sortie utilise $O(q)$.

Construire l'ensemble à chaque tour répéterait q fois le prétraitement : on retrouverait un coût $O(qn)$ moyen.

Quel prétraitement vaut la peine ?

À vous. Un catalogue contient un million d'identifiants. On reçoit dix mille questions d'appartenance. Comparez trois stratégies : parcours répétés, copie triée puis dichotomies, ensemble construit une fois.

Stratégie	Préparation	Coût de q questions	Mémoire auxiliaire
Parcours	...	...	...
Tri et dichotomies	...	...	...
Ensemble	...	...	...

Votre choix change-t-il si l'on ne pose qu'une question ? Si la liste est déjà triée ? Si l'on veut aussi récupérer des valeurs dans un intervalle ordonné ?

On demande ici seulement la présence d'une valeur, pas son indice dans la liste d'origine.

Le choix dépend des questions à poser

Stratégie	Préparation	Questions	Auxiliaire
Parcours	Aucune	$O(qn)$ au pire	$O(1)$
Copie triée	$O(n \log n)$ au pire	$O(q \log n)$ au pire	$O(n)$
Ensemble	$O(n)$ moyen	$O(q)$ moyen	$O(n)$

Les parcours peuvent examiner 10^{10} valeurs. Les dichotomies consultent au plus 20 valeurs par question, après le tri. L'ensemble évite les parcours répétés pour l'appartenance seule.

Pour une seule question, un parcours évite de créer un index. Sur une liste déjà triée, la dichotomie utilise $O(1)$ auxiliaire. L'ordre trié sert aussi aux recherches d'intervalles. La sortie de q booléens, si on la conserve, ajoute $O(q)$ dans les trois cas.

Une mesure sur votre machine

À vous. Réutilisez `doublon_paires` et `doublon_ensemble`. Le code de mesure ci-dessous est fourni : lancez-le, puis comparez l'évolution quand n double.

```
from timeit import repeat

for n in [200, 400, 800]:
    valeurs = list(range(n))
    a = min(repeat(lambda: doublon_paires(valeurs),
                  number=3, repeat=3)) / 3
    b = min(repeat(lambda: doublon_ensemble(valeurs),
                  number=3, repeat=3)) / 3
    print(n, round(a * 1000, 3), round(b * 1000, 3))
```

Colonnes : n , paires en millisecondes, ensemble en millisecondes. Pourquoi utiliser des valeurs toutes distinctes ? La construction de l'ensemble est-elle incluse dans la mesure ?

Source : Documentation Python : `timeit`

Lire une mesure sans lui faire dire trop

Un exemple d'exécution avec Python 3.12.14, sur la machine de préparation :

n	Paires (ms par appel)	Ensemble (ms par appel)
200	0.164	0.003
400	0.903	0.007
800	3.977	0.011

Vos durées peuvent être différentes. Chaque mesure retient le minimum de trois séries, chacune contenant trois appels ; on divise par trois pour obtenir une durée par appel.

Les valeurs sont distinctes : aucun doublon n'interrompt les parcours. La fonction avec ensemble inclut bien sa construction. La liste d'entrée est construite avant la mesure.

Le comptage donne 19 900, 79 800 et 319 600 paires : presque quatre fois plus à chaque doublement. Une mesure bruitée sur trois tailles ne prouve pas à elle seule une classe de complexité.

Deux prix pour un budget

À vous. Écrivez `deux_prix_paires(prix, budget)`. Les prix et le budget sont des entiers positifs ou nuls.

- Renvoyer un tuple de deux **indices distincts** dont les prix totalisent le budget.
- S'il existe plusieurs paires, n'importe laquelle convient ; sans solution, renvoyer `None`.
- Ne pas modifier la liste. Deux articles peuvent avoir le même prix.

```
prix = [8, 3, 6, 4]
budget = 10
# Écrire une première version avec deux boucles.
# Tester aussi [5] avec 10, puis [5, 5] avec 10.
```

Reprenez les paires $i < j$. Dans votre trace, notez les indices, leur somme et la décision.

Deux prix : la version directe

```
def deux_prix_paires(prix, budget):  
    for i in range(len(prix)):  
        for j in range(i + 1, len(prix)):  
            if prix[i] + prix[j] == budget:  
                return i, j  
    return None  
  
print(deux_prix_paires([8, 3, 6, 4], 10))  
print(deux_prix_paires([5], 10))  
print(deux_prix_paires([5, 5], 10))
```

Affichages : (2, 3), puis None, puis (0, 1).

Les sommes du premier appel sont 11, 14, 12, 9, 7, puis 10. On renvoie les indices 2 et 3, dont les prix sont 6 et 4. Pire cas : $O(n^2)$ en temps; $O(1)$ auxiliaire.

Chercher uniquement le prix manquant

À vous. Pour un prix courant x , le seul prix utile est $\text{budget} - x$. Écrivez `deux_prix_index(prix, budget)` avec un dictionnaire associant chaque prix déjà vu à un indice.

1. Écrivez le pseudocode en précisant l'ordre du test et de l'ajout au dictionnaire.
2. Tracez `[8, 3, 6, 4]` avec le budget 10 : indice, prix, complément et dictionnaire.
3. Codez, puis comparez les résultats aux tests de la version directe.

Le dictionnaire doit permettre de renvoyer les *positions* des deux articles. Un ensemble de prix suffirait-il pour connaître ces indices ?

Ne trie pas les prix : les indices demandés sont ceux de la liste d'origine.

Le dictionnaire des positions : correction

```
def deux_prix_index(prix, budget):
    positions = {}
    for i, valeur in enumerate(prix):
        complement = budget - valeur
        if complement in positions:
            return positions[complement], i
        positions[valeur] = i
    return None

print(deux_prix_index([8, 3, 6, 4], 10))
print(deux_prix_index([5], 10))
print(deux_prix_index([5, 5], 10))
```

Affichages : (2, 3), puis None, puis (0, 1).

Le dictionnaire contient des indices antérieurs à i : les deux positions sont distinctes. Temps moyen $O(n)$; espace auxiliaire $O(n)$ au pire. Un ensemble seul ne mémoriserait pas les indices.

Une erreur qui réutilise le même article

À vous. Dans cette variante, l'ajout a été déplacé avant le test.

```
def deux_prix_bug(prix, budget):  
    positions = {}  
    for i, valeur in enumerate(prix):  
        positions[valeur] = i  
        complement = budget - valeur  
        if complement in positions:  
            return positions[complement], i  
    return None
```

Sans exécuter, prévoyez `deux_prix_bug([5], 10)`. Quelle règle du contrat ce résultat viole-t-il ? Déplacez une seule ligne pour corriger la fonction.

Vérifiez ensuite que `[5, 5]` avec le budget 10 reste un cas valide.

Des positions déjà vues, toujours distinctes

La version erronée renvoie (0, 0) pour [5], car elle retrouve le prix qu'elle vient d'ajouter. La correction teste le complément **avant** $\text{positions}[\text{valeur}] = i$.

Trace de la version correcte sur [8, 3, 6, 4], budget 10 :

i	Prix	Complément	Dictionnaire avant	Décision
0	8	2	{}	Ajouter 8 : 0.
1	3	7	{8: 0}	Ajouter 3 : 1.
2	6	4	{8: 0, 3: 1}	Ajouter 6 : 2.
3	4	6	{8: 0, 3: 1, 6: 2}	Renvoyer (2, 3).

Sur [5, 5], le premier 5 est mémorisé à l'indice 0 ; le second le retrouve à l'indice 1. Deux prix égaux sont autorisés, à condition d'appartenir à deux positions différentes.

Deux pointeurs sur des prix déjà triés

À vous. Les prix sont maintenant déjà triés : [2, 3, 6, 8, 11]. Le budget vaut 10. Placez un indice à chaque extrémité et calculez la somme des deux prix.

- Si la somme est trop grande, quel indice faut-il déplacer ?
- Si elle est trop petite, quel indice faut-il déplacer ?
- Pourquoi faut-il s'arrêter avant que les deux indices se rejoignent ?

Tracez les déplacements, puis écrivez `deux_prix_tries(prix, budget)`. Renvoyez deux indices distincts ou `None`, sans créer de dictionnaire.

Le contrat porte ici sur les indices de la liste *déjà triée* reçue en argument. Trier une liste quelconque auparavant changerait la signification des indices.

Deux pointeurs : correction

```
def deux_prix_tries(prix, budget):
    gauche = 0
    droite = len(prix) - 1
    while gauche < droite:
        total = prix[gauche] + prix[droite]
        if total == budget:
            return gauche, droite
        if total < budget:
            gauche = gauche + 1
        else:
            droite = droite - 1
    return None

print(deux_prix_tries([2, 3, 6, 8, 11], 10))
```

On teste $(0, 4) : 2 + 11 = 13$, puis $(0, 3) : 2 + 8 = 10$. L'affichage est $(0, 3)$. Chaque tour rapproche les bornes d'une position : au plus $n - 1$ tours, temps $O(n)$ et auxiliaire $O(1)$.

Des moyennes sur trois valeurs voisines

À vous. Pour $[2, 4, 6, 8, 10]$ et une largeur $k = 3$, calculez la moyenne de chaque groupe de trois valeurs consécutives.

Fenêtre	Somme	Moyenne
$[2, 4, 6]$	...	...
$[4, 6, 8]$	...	...
$[6, 8, 10]$	...	...

Écrivez `moyennes_directes(valeurs, k)` en recalculant la somme de chaque tranche. Vous pouvez utiliser `sum()` et `valeurs[debut:debut+k]`.

Contrat : valeurs entières, k entier avec $1 \leq k \leq n$; renvoyer une liste de moyennes, sans modifier l'entrée. Si k est hors de cet intervalle, lever `ValueError`. Quel est le dernier indice de départ possible ?

Le recalcul de chaque fenêtre

```
def moyennes_directes(valeurs, k):  
    if k <= 0 or k > len(valeurs):  
        raise ValueError("largeur invalide")  
    resultat = []  
    for debut in range(len(valeurs) - k + 1):  
        total = sum(valeurs[debut:debut + k])  
        resultat.append(total / k)  
    return resultat  
  
print(moyennes_directes([2, 4, 6, 8, 10], 3))
```

Résultat

```
[4.0, 6.0, 8.0]
```

Les sommes sont 12, 18 et 24. Le dernier départ est $n - k$, inclus : il existe $n - k + 1$ fenêtres. Chaque tranche contient k valeurs à copier puis à sommer.

La somme de la fenêtre suivante

À vous. On connaît déjà la somme de $[2, 4, 6]$: elle vaut 12. Quand la fenêtre devient $[4, 6, 8]$, quelles valeurs sortent et entrent ?

Nouvelle fenêtre	Sortante	Entrante	Nouvelle somme
$[4, 6, 8]$	...	...	...
$[6, 8, 10]$	...	...	...

Écrivez une formule utilisant la somme précédente. Puis rédigez le pseudocode de toute la fonction : comment calculer la première somme, stocker sa moyenne et parcourir les valeurs suivantes ?

Si `droite` est l'indice de la valeur qui entre dans une fenêtre de largeur `k`, quel est l'indice de la valeur qui en sort ?

La mise à jour de la somme : correction

nouvelle somme = ancienne somme – valeur sortante + valeur entrante

droite	Sortante	Entrante	Somme après	Moyenne
3	2	8	$12 - 2 + 8 = 18$	6.0
4	4	10	$18 - 4 + 10 = 24$	8.0

La première moyenne, 4.0, a été stockée avant ces deux mises à jour.

Vérifier la largeur k

Calculer la somme des k premières valeurs

Créer la liste contenant la première moyenne

POUR droite allant de k à $n - 1$ inclus

Retirer la valeur d'indice droite - k de la somme

Ajouter la valeur d'indice droite à la somme

Ajouter la nouvelle moyenne à la liste

RENVoyer la liste

Une moyenne mobile sans tranches

À vous. Traduisez le pseudocode en `moyennes_glissantes(valeurs, k)`. Conservez le contrat et les résultats de `moyennes_directes`.

- Calculez la première somme avec une boucle sur les indices 0 à $k - 1$.
- Ajoutez la première moyenne au résultat avant la boucle de déplacement.
- Parcourez ensuite les indices k à $n - 1$ pour mettre à jour la somme.

N'utilisez pas de tranche : on veut éviter de créer une copie temporaire de k valeurs.

```
# Comparer les deux versions sur ces données.  
valeurs = [2, 4, 6, 8, 10]  
k = 3  
# moyennes_glissantes(valeurs, k)  
# moyennes_directes(valeurs, k)
```

Avec $k = n$, combien de tours la seconde boucle doit-elle faire ?

La fenêtre glissante : correction

```
def moyennes_glissantes(valeurs, k):  
    if k <= 0 or k > len(valeurs):  
        raise ValueError("largeur invalide")  
    total = 0  
    for i in range(k):  
        total = total + valeurs[i]  
    resultat = [total / k]  
    for droite in range(k, len(valeurs)):  
        total = total - valeurs[droite - k]  
        total = total + valeurs[droite]  
        resultat.append(total / k)  
    return resultat  
  
print(moyennes_glissantes([2, 4, 6, 8, 10], 3))
```

L'affichage est `[4.0, 6.0, 8.0]`. Si $k = n$, la seconde boucle ne fait aucun tour ; la première moyenne constitue déjà toute la sortie.

Les tests qui protègent la fenêtre

À vous. Prévoyez les résultats, puis écrivez les assertions pour les cas valides. Pour les cas invalides, vérifiez que `ValueError` est bien levée.

Valeurs	k	Résultat ?	Ce que le cas vérifie
[2, 4, 6]	1	...	Une seule valeur par fenêtre
[2, 4, 6]	3	...	Une seule fenêtre au total
[-2, 0, 2]	2	...	Valeurs négatives
[2, 4]	0	...	Largeur nulle
[2, 4]	3	...	Largeur supérieure à la liste
[]	1	...	Aucune fenêtre possible

Ajoutez un test de largeur négative et vérifiez que l'entrée reste intacte. Expliquez pourquoi renvoyer une liste vide pour une largeur invalide changerait le contrat.

Les fenêtres valides : correction

```
assert moyennes_glissantes([2, 4, 6], 1) == [2.0, 4.0, 6.0]
assert moyennes_glissantes([2, 4, 6], 3) == [4.0]
assert moyennes_glissantes([-2, 0, 2], 2) == [-1.0, 1.0]

entree = [2, 4, 6]
avant = entree.copy()
moyennes_glissantes(entree, 2)
assert entree == avant
```

Ces assertions passent sans affichage. Pour $k = 1$, les moyennes retrouvent les valeurs de départ. Pour $k = n$, il n'existe qu'une fenêtre. Les valeurs négatives ne changent pas la règle de calcul.

Le dernier test vérifie un effet du programme : la fonction renvoie une nouvelle liste et laisse l'entrée intacte.

Les largeurs invalides : correction

```
for valeurs, k in [( [2, 4], 0), ([2, 4], 3),  
                  ([], 1), ([2, 4], -1)]:  
    try:  
        moyennes_glissantes(valeurs, k)  
    except ValueError:  
        print("largeur refusée")  
    else:  
        raise AssertionError("largeur acceptée à tort")
```

largeur refusée s'affiche quatre fois. Le else du try ne s'exécute que si aucune exception n'a été levée : il signifierait qu'une largeur invalide a été acceptée.

Le contrat distingue un résultat valide d'une largeur pour laquelle le calcul n'est pas défini.

Ce que la fenêtre permet d'économiser

	Recalcul avec tranches	Mise à jour sans tranches
Travail	$(n - k + 1)$ sommes de k valeurs	k ajouts, puis $n - k$ mises à jour
Temps	$O((n - k + 1)k)$, donc $O(nk)$	$O(n)$
Auxiliaire hors sortie	$O(k)$ pour la tranche temporaire	$O(1)$
Sortie	$n - k + 1$ moyennes	$n - k + 1$ moyennes

Avec $n = 1000$ et $k = 100$, le recalcul additionne 90 100 valeurs. La mise à jour fait 100 ajouts initiaux, puis 900 soustractions et 900 ajouts.

Si k reste constant, les deux temps sont linéaires en n ; la mise à jour réduit le travail répété. Si k grandit avec n , le gain peut aussi changer l'ordre de croissance.

Un petit journal d'événements

À vous. On reçoit des couples (utilisateur, action) :

```
evenements = [  
    ("u1", "vue"),  
    ("u2", "achat"),  
    ("u1", "achat"),  
    ("u2", "achat"),  
]
```

Écrivez `resumer_evenements(evenements)`. En un seul parcours, la fonction doit renvoyer :

1. un dictionnaire donnant le nombre d'apparitions de chaque action ;
2. l'ensemble des utilisateurs rencontrés ;
3. un dictionnaire donnant, pour chaque acheteur, la position de son **premier** achat.

Prévoyez les trois résultats, puis rédigez le pseudocode et codez. Les indices commencent à 0 ; le second achat de u2 ne doit pas remplacer le premier.

Trois résumés dans le même parcours

```
def resumer_evenements(eventements):
    comptes = {}
    utilisateurs = set()
    premiers = {}
    for i, (utilisateur, action) in enumerate(eventements):
        utilisateurs.add(utilisateur)
        comptes[action] = comptes.get(action, 0) + 1
        if action == "achat" and utilisateur not in premiers:
            premiers[utilisateur] = i
    return comptes, utilisateurs, premiers

r = resumer_evenements(eventements)
print(r[0], sorted(r[1]), r[2], sep="\n")
```

`get(action, 0)` permet le premier comptage. Le test `utilisateur not in premiers` empêche de remplacer la position du premier achat. Le tri sert uniquement à rendre l'affichage des utilisateurs lisible.

Le journal : résultats et trace

Résultat

```
{ 'vue': 1, 'achat': 3 }  
[ 'u1', 'u2' ]  
{ 'u2': 1, 'u1': 2 }
```

Position	Événement	Comptage mis à jour	Premiers achats après le tour
0	u1, vue	vue : 1	{}
1	u2, achat	achat : 1	{u2: 1}
2	u1, achat	achat : 2	{u2: 1, u1: 2}
3	u2, achat	achat : 3	Inchangé

La fonction seule prend $O(n)$ en moyenne. Pour u utilisateurs et a actions distinctes, ses résultats occupent $O(u + a)$; hors résultats, l'auxiliaire est $O(1)$. Une entrée vide renvoie deux dictionnaires vides et un ensemble vide.

Quatre affirmations à discuter

À vous. Pour chaque affirmation, choisissez vrai ou faux et donnez une raison ou un contre-exemple.

1. Deux boucles écrites dans une fonction signifient toujours un temps $O(n^2)$.
2. Un test d'appartenance dans un ensemble dispense de compter le coût de sa construction.
3. Une recherche dichotomique peut être appliquée directement à n'importe quelle liste.
4. La moyenne glissante présentée utilise $O(1)$ auxiliaire, même si sa liste de résultats grandit.

Puis complétez cette phrase pour une fonction du chapitre :

« Pour une entrée de taille ..., dans le cas ..., le temps est ... parce que L'espace auxiliaire est ...; la sortie occupe »

Les affirmations : correction

1. **Faux.** Deux parcours successifs coûtent $O(n)$ au total. On compte les répétitions, pas seulement les mots `for` ou `while`.
2. **Faux.** Construire l'ensemble de n valeurs coûte $O(n)$ en moyenne et jusqu'à $O(n)$ en mémoire.
3. **Faux.** La liste doit être triée ; sinon on risque d'écarter une zone qui contient la cible.
4. **Vrai, avec notre convention.** La somme et les indices prennent $O(1)$ auxiliaire. La liste des moyennes est la sortie : elle occupe $O(n - k + 1)$.

Exemple d'analyse : pour n prix, la recherche d'une paire avec un dictionnaire prend $O(n)$ en moyenne, car chaque prix déclenche un travail moyen borné. Le dictionnaire utilise $O(n)$ auxiliaire au pire ; la sortie contient au plus deux indices, donc $O(1)$.

Les repères pour choisir une solution

Situation	Solution travaillée	Condition ou coût à garder en tête
Recherche sans ordre	Parcours linéaire	$O(n)$ au pire, aucun index à créer
Liste déjà triée	Dichotomie	$O(\log n)$ au pire, ordre garanti
Présence d'un doublon	Ensemble	$O(n)$ moyen, mémoire $O(n)$
Deux prix pour une cible	Dictionnaire	Complément testé avant l'ajout
Fenêtres voisines	Somme mise à jour	$O(n)$, compter la sortie à part
Requêtes répétées	Index construit une fois	Compter construction et requêtes

Avant de garder une nouvelle version, vérifiez les mêmes cas ordinaires et les mêmes limites. Un changement de méthode doit préserver le résultat attendu et les effets autorisés sur l'entrée.

Source : MIT 6.100L ; notes du cours, chapitre 3